🗓️ Day 2 — SSH Hardening & Key-Based Authentication

Focus day. Full lab.

📌 Topic

SSH is the front door to every Linux server. Default config is a liability. Today you lock it down to production standard — the way every serious company expects it configured before a server goes live.

💻 Commands

```
# Install , start & enable openssh-erver on ubuntu
sudo apt install openssh-server -y
sudo systemctl start  sshd.service
sudo systemctl enable  sshd.service

# Check current SSH config (know what's running before touching it)
sshd -T | grep -E 'permitrootlogin|passwordauthentication|port|allowusers|maxauthtries'

# Test config BEFORE restarting (never restart blind)
sshd -t

# Reload safely (not restart — keeps existing sessions alive)
systemctl reload sshd

# Generate Ed25519 key pair (stronger than RSA 2048)
ssh-keygen -t ed25519 -C "prod-server-$(hostname)-$(date +%Y%m%d)"

# Copy public key to server
ssh-copy-id -i ~/.ssh/id_ed25519.pub user@server

# Check who has authorized keys (audit)
find /home -name "authorized_keys" -exec ls -la {} \;
find /root -name "authorized_keys" -exec ls -la {} \;

# Watch live auth attempts
tail -f /var/log/auth.log | grep sshd        # Debian
tail -f /var/log/secure | grep sshd          # RHEL


# Check failed login count per IP
grep "Failed password" /var/log/auth.log | awk '{print $11}' | sort | uniq -c | sort -rn | head -20
```

🧪 Lab Task

Do this in order. Do not skip steps.

Step 1 — Audit current SSH state

```
sshd -T | grep -E 'permitrootlogin|passwordauthentication|pubkeyauthentication|port|maxauthtries|logingracetime'
```

Write down what each value currently is. Know your baseline before changing anything.

Step 2 — Generate a proper key pair

```
ssh-keygen -t ed25519 -C "lab-vm-$(date +%Y%m%d)" -f ~/.ssh/lab_ed25519
```

Set a passphrase. Yes, always.

Step 3 — Deploy the key

```
# If testing on same machine (or second VM)
ssh-copy-id -i ~/.ssh/lab_ed25519.pub user@target-ip

# Manual method (know both ways)
cat ~/.ssh/lab_ed25519.pub >> ~/.ssh/authorized_keys
chmod 600 ~/.ssh/authorized_keys
chmod 700 ~/.ssh/
```

Step 4 — Harden `/etc/ssh/sshd_config`

```
sudo cp /etc/ssh/sshd_config /etc/ssh/sshd_config.bak.$(date +%Y%m%d)
sudo nano /etc/ssh/sshd_config
```

Set these values:

```
Port 2222                # Non-default port
PermitRootLogin no       # Never root direct login
PasswordAuthentication no   # Keys only
PubkeyAuthentication yes
MaxAuthTries 3
LoginGraceTime 20
AllowUsers youruser      # Whitelist only

X11Forwarding no
AllowTcpForwarding no
Banner /etc/ssh/banner     # Legal warning banner
```

Step 5 — Validate before applying

```
sudo sshd -t   # Must return clean. If errors, fix before continuing.
```

Step 6 — Open new terminal, test key login FIRST

```
ssh -i ~/.ssh/lab_ed25519 -p 2222 user@server
```

Only reload sshd AFTER you confirm key login works. This is how you avoid locking yourself out.

bash

```
sudo systemctl reload sshd
```

Step 7 — Verify hardening

```
sshd -T | grep -E 'permitrootlogin|passwordauthentication|maxauthtries|port'
```

🔐 Security Angle

How an attacker exploits default SSH:

- Brute-force `root` with `password authentication on` → common tool: `hydra`, `medusa`
- Scan for port 22 → immediate targeting by automated bots (watch your logs, you'll see this within hours of spinning up a VPS)
- Steal a weak private key (RSA 1024 is broken, RSA 2048 is marginal — Ed25519 is current standard)
- Plant an SSH key in `authorized_keys` for persistence (this is exactly what Q5 yesterday was about)

What logs reveal it:

```
grep "Failed password" /var/log/auth.log | wc -l     # volume of brute force
grep "Accepted publickey" /var/log/auth.log          # successful key logins
grep "Invalid user" /var/log/auth.log | head -20     # username enumeration attempts
```

How to detect:

```
# IPs with 10+ failed attempts
grep "Failed password" /var/log/auth.log | awk '{print $11}' | sort | uniq -c | sort -rn | awk '$1 > 10'
```

How to prevent:

- Keys only, no passwords
- `fail2ban` or `sshguard` (we configure this in Level 3)
- Port knocking or VPN-only SSH for high-security environments
- `AllowUsers` whitelist — unknown users can't even attempt auth

🔥 Challenge Task

No hints.

> Your junior colleague deployed a server with default SSH config. Root login is enabled, password auth is on, port 22. At 2 AM you get an alert — 3,000 failed SSH attempts in 10 minutes, then one successful `root` login from an IP in a country your company has no presence in.

Write a full incident response playbook for this scenario — step by step, in order. Include: containment, investigation, remediation, and what you put in the post-mortem.

🚀 Real-World Scenario

You're onboarding 3 new developers. Company policy: SSH key access only, no passwords, each dev gets their own key, root login forbidden, keys must be rotated every 90 days.

Your job:

1. How do you deploy 3 different keys to the same server without overwriting each other?
2. How do you revoke access for one developer without affecting the others?
3. How do you enforce 90-day rotation — can it be automated?

Think through this. We discuss your answers.

❓ Quick Quiz

Q1. What's the difference between `systemctl restart sshd` and `systemctl reload sshd` — and why does it matter in production?

Q2. You set `PasswordAuthentication no` but a user can still log in with a password. What's the first place you check?

Q3. Why is Ed25519 preferred over RSA 2048 for new key generation?

Q4. What permission should `~/.ssh/authorized_keys` be set to — and what happens if it's wrong?

Q5. An attacker gets read access to your server. Your private key is stored at `~/.ssh/id_ed25519`. Are they in?

📁 GitHub Task

Repo: `linux-server-hardening`

Start this today. You'll build it across multiple days.

```
linux-server-hardening/
├── README.md
├── ssh/
│   ├── sshd_config.hardened        # Your hardened config (strip sensitive values)
│   ├── ssh-hardening-checklist.md  # Step by step checklist
│   └── ssh-audit-commands.sh       # Script: audits current SSH state vs baseline
├── scripts/
│   └── (future scripts go here)
```

`ssh-audit-commands.sh` should:

- Check current `sshd -T` values against expected hardened values
- Output PASS/FAIL per setting
- Exit code 1 if any setting fails (so it can be used in CI/CD or cron)

README must include:

- Why each setting was changed
- How to test before applying
- How to safely reload without dropping sessions

➡️ Day 3 Preview

Topic: File Permissions, Ownership & ACLs

You'll learn why `chmod 777` is a firing offense, what SUID/SGID bits actually do to your attack surface, and how to find every dangerous permission on a live system.

Come ready to answer:

> *"What does the sticky bit do — and name one directory on every Linux system where it's set by default and why."*

Quiz answers when you're ready. Challenge task is homework — bring it next session.